

An Autoencoder and Double Random Forest-Based Intrusion Detection System for Unknown Attack Defense

Chen Zhang¹, Lu Zheng², Huakun Huang³, *Member, IEEE*, and Chunhua Su⁴, *Member, IEEE*

Abstract—With the rapid advancement of technology and the growing prevalence of connected vehicle architectures, security concerns in vehicle networks have become increasingly critical, the vehicles are high-value assets. Thus, unauthorized intrusions can pose substantial financial and safety risks. Although many researchers have employed deep learning techniques for vehicle network intrusion detection, existing methods often struggle to efficiently extract network traffic features and handle unknown or evolving malicious attacks. In this paper, we propose a novel Internet of Vehicles intrusion detection system (IDS) that combines an autoencoder (AE) with a double random forest (DRF). Our two-layer architecture is designed to accurately distinguish between unknown attacks, known attacks, and normal traffic. We evaluated the performance of the proposed DRF algorithm against six traditional machine learning and deep learning methods and compared it with relevant state-of-the-art solutions. The experimental results demonstrate that our method outperforms existing approaches.

Index Terms—Stratified K-fold, autoencoder, double random forest, IDS, IoV.

I. INTRODUCTION

THE Internet of Vehicles (IoV) is an advanced network architecture that integrates vehicles, pedestrians, transportation infrastructure, and cloud computing resources through an Internet-based platform. It facilitates real-time information exchange and intelligent decision-making by leveraging mobile communications, sensing technology, data processing, and artificial intelligence [1], [2]. This integration enhances traffic management, road safety, and overall transportation efficiency. Modern telematics heavily relies on 5G and future 6G networks to

enable low-latency communication, which supports applications such as autonomous driving, intelligent transportation systems (ITS), and in-vehicle entertainment [3]. By combining human driving behavior with machine intelligence, IoV contributes significantly to the development of ITS [4], [5]. IoV relies on advanced networks such as 5G to achieve efficient communication between vehicles and infrastructure, perceives the environment through the fusion of multiple sensors, and combines artificial intelligence and Big Data to optimize decision-making and ensure data security and user privacy.

IoV has been deployed in various smart cities where vehicles collect and transmit traffic data via onboard sensors and connected devices [6], cloud servers process these data and provide real-time insights used to enhance smart city operations through traffic data collection [7], [8], improve traffic management by adjusting traffic lights and optimizing route planning [9], support autonomous driving safety and decision-making with real-time environmental data [3] and enable in-vehicle entertainment and connectivity features [10], [11]. However, alongside these benefits, IoV faces critical security threats due to the open nature of vehicle networks, which makes them vulnerable to malicious attacks.

Despite advancements in the IoV domain, cybersecurity remains a paramount concern since vehicles, as high value mobile nodes, expose a broad and shifting attack surface across V2V (Vehicle-to-Vehicle) and V2I (Vehicle-to-Infrastructure) channels [12]. The heterogeneous mix of onboard sensors, ECUs (Electronic Control Unit), and wireless links creates multiple vulnerabilities from spoofing and man-in-the-middle to denial-of-service that adversaries can exploit as vehicles traverse different network domains. In 2024, researchers exploited a Kia web portal flaw to remotely unlock, track, and start millions of vehicles using only licenseplate scans; similarly, attackers leveraged a Man-in-the-Middle vulnerability at Tesla charging stations to capture owner credentials and added unauthorized phone keys, granting remote access and ignition. Traditional IDSs, rooted in static rule sets, struggle to keep pace with novel and rapidly evolving attack vectors, rendering them ineffective against zero-day threats [13], [14]. Even IDS approaches leveraging deep learning often falter when confronted with previously unseen malicious patterns, generating high rates of false positives and false negatives that undermine trust in their alerts [15], [16]. Compounding these challenges are the real-time processing

Received 10 February 2025; revised 9 May 2025, 25 June 2025, and 28 July 2025; accepted 19 September 2025. Date of publication 23 September 2025; date of current version 6 March 2026. This work was supported in part by the Japan Society for the Promotion of Science Grant-in-Aid for Scientific Research (C) under Grant 23K11103, in part by the Natural Science Foundation of Guangdong Province, China under Grant 2025A1515011755, in part by the Key Laboratory of Cognitive Radio and Information Processing, Ministry of Education under Grant CRKL240204, and in part by SCAT Foundation under Grant for Researchers. The review of this article was coordinated by Dr. Jun Cai. (Corresponding author: Chunhua Su.)

Chen Zhang and Chunhua Su are with the Department of Computer Science and Engineering, The University of Aizu, Aizuwakamatsu 965-8580, Japan (e-mail: chsu@u-aizu.ac.jp).

Lu Zheng is with the School of Computer and Communication, Lanzhou University of Technology, Lanzhou 730050, China.

Huakun Huang is with the School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou 510006, China.

Digital Object Identifier 10.1109/TVT.2025.3613343

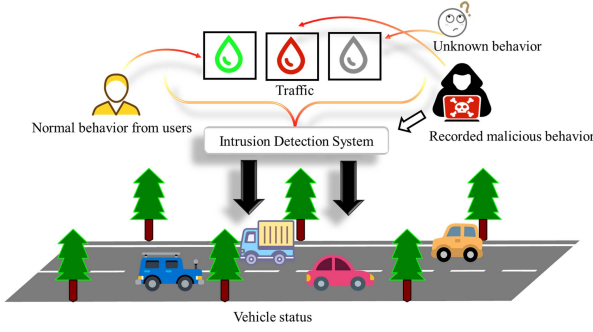


Fig. 1. Traffic composition in the IoV.

requirements and resource constraints of in-vehicle hardware, which limit the deployment of more complex or compute-intensive security mechanisms, leaving intelligent transportation systems exposed to increasingly sophisticated cyber threats.

To address these challenges, we propose an adaptive IDS for the IoV that continuously learns from network traffic, via a dynamic unknown attack detection module to uncover novel threats beyond the reach of traditional rule-based IDSs, refines known-attack detection by deeply analyzing vehicle behavior and employing a hierarchical classification mechanism and implements a multi-layer defense framework categorizing traffic into normal, known malicious and unknown malicious, as shown in Fig. 1, with specialized detection mechanisms at each layer to overcome single-layer limitations and ensure robust real-time security for intelligent transportation systems. The following is our contribution:

- We propose CAIDS (CDDRF-AE—IDS), which consists of an AE module for detecting unknown traffic and a CDDRF model for detecting known traffic, which can effectively identify all traffic in open scenarios with high detection capability.
- We use AE that is actively updated to detect unknown traffic and update the training set using the normal traffic finally filtered by CAIDS to improve the detection accuracy, which effectively avoids the impact of unknown traffic on IoV.
- We propose CDDRF (Confidence Driven Dual Random Forest), which utilizes hierarchical k-folding to classify the training set into a high-confidence dataset and a low-confidence dataset based on the confidence threshold r . We obtain two random forest models of a^r, b^r , which are voted on to get the best result. The actual ability of the model is verified by several comparisons and far exceeds current state-of-the-art algorithms.

The rest of this paper is structured as follows: Section II reviews related research on intrusion detection in IoV. Section III presents the CAIDS framework and its operation. Section IV explains the AE model for unknown attack detection. Section V details the theoretical model of the CDDRF classifier. Section VI provides experimental evaluations and performance analysis. Section VII concludes the study.

II. RELATED WORK

This section discusses the research on IoV intrusion detection in recent years, especially the detection of unknown and known

attacks, and compares the differences between our research and other algorithms.

In [17] proposed TLOC, which achieves efficient intrusion detection by optimizing CNN (Convolutional Neural Network) and transfer learning, but it is mainly aimed at static environments and lacks the ability to process and dynamically update unknown traffic. Our research actively detects unknown traffic and dynamically optimizes training data, significantly improving its adaptability to open scenarios. [18] proposed LIDS, an LSTM-based IDS for detecting forged emergency messages in vehicular ad hoc networks (VANETs). The core approach is to collect the time series of traffic parameters of vehicles, use LSTM (Long Short-Term Memory) classifiers to identify traffic event patterns, and judge the authenticity of emergency messages by time series features. However, their method mainly relies on historical data training. At the same time, insufficient data quality optimization may lead to performance degradation under high noise or imbalanced data. [19] proposed NIDS, which balanced the dataset through a hybrid sampling algorithm (combining ADASYN and RENN) and selected important features using random forest and Pearson correlation analysis. The model extracted spatial features through CNN and combined them with BiGRU to capture long-distance dependency information and achieve efficient classification of intrusion behaviors. However, they only focused on the classification of known traffic and did not perform hierarchical optimization on data quality.

In [20] proposed AUIDS, a deep learning automatic update IDS based on cloud-edge-car collaboration, which can detect known and unknown attacks in open scenarios. The system extracts traffic features through 1D-CNN and multi-head attention mechanism, and processes unknown attack traffic through two-layer filters and clustering algorithm, generating new data sets to update attack identifiers, thereby improving detection accuracy and adaptability. In contrast, their research relies on cloud support for update efficiency, and the optimization of data quality and system performance is relatively insufficient. [21] proposed HDA-IDS, which is centered on an innovative anomaly detection model CL-GAN, which combines CNN-LSTM with GAN (Generative Adversarial Network) to establish a normal behavior baseline. Experiments show that HDA-IDS improves the performance of other methods by an average of 5% on NSL-KDD, CICIDS2018, and Bot-IoT datasets. However, this method has weak adaptability to unknown traffic in dynamic scenarios and lacks a data quality optimization mechanism. In addition, HDA-IDS relies more on baseline data generated by GAN and may be insufficiently robust to large-scale complex traffic in open scenarios.

In [22] proposed SupEnML, which integrates a supervised learning framework with a feature selection method, with the goal of optimizing detection performance by minimizing false positives and improving accuracy. However, this method still mainly focuses on static data analysis and lacks dynamic detection capabilities and active adaptability for unknown traffic. [23] proposed IoT-PRIDS, which establishes a baseline of normal device behavior through feature mapping of network packets to detect anomalies and potential intrusions. The model uses packet header fields to create “representations”, which effectively

reduces data complexity and can detect anomalies in IoT (Internet of Things) environments in real-time with low computational overhead. However, IoT-PRIDS only models normal traffic and lacks an update mechanism for unknown flows. Its lightweight design also results in poor adaptability to complex attacks and subpar performance.

In [24] proposed ENCSA-PSO, which combines the search strategy advantages of CSA (Cuckoo Search Algorithm) and the fast convergence ability of PSO (Particle Swarm Optimization), selects the global best features from a large search space, and classifies these features using a random forest classifier, while eliminating feature bias through weighted minimum mean square processing. This approach focuses mainly on feature selection and classification problems, but lacks adaptability to handle dynamic traffic. [25] proposed HyperVision which captures encrypted traffic interaction patterns using a memory graph and unsupervised graph learning based on connectivity, sparsity and statistical features. This enables detection of malicious flows without labeled datasets but the model cannot update dynamically to accommodate shifting traffic characteristics and often hits resource bottlenecks in large-scale environments. CAIDS addresses these challenges by actively refreshing its training set and introducing a layered data quality optimization framework that balances processing efficiency with robust detection accuracy across diverse and evolving traffic scenarios. [26] proposed MLMCS, a framework that includes four steps: data preprocessing, feature space partitioning to reduce model complexity, hierarchical binary classification of each feature subspace based on a multilayer classifier system (MLCS) based on a GMDH neural network, and merging classification results based on the Dempster-Shafer theory to reduce overfitting. However, the MLMCS model is complex, requires the feature space to be divided into multiple subspaces, and relies on multi-layer classifiers, and the computational cost may be high.

Existing IDSs are generally designed for static environments, lack the ability to adapt to dynamic changes and unknown traffic, have insufficient data quality optimization, rely on historical data for training, and possess complex models with high computational overhead. To address these challenges, we propose CAIDS, an integrated adaptive IDS. CAIDS actively detects unknown traffic through the AE module and efficiently identifies known traffic in combination with the CDDRF model. Additionally, stratified k-fold cross-validation is employed to optimize data quality, and the voting mechanism of the dual random forest model is utilized to improve detection accuracy and robustness. The system performs well in open and dynamic network environments, significantly enhancing the adaptability, accuracy, and efficiency of intrusion detection while overcoming many shortcomings of existing methods.

III. SYSTEM DESIGN

This section introduces in-depth how CAIDS solves the problem of unknown attacks, such as zero-day attacks, which are highly harmful to the IoV, and the problem of adversaries using various known attacks to invade the IoV architecture. We explain

the CAIDS framework, which includes the identification of normal traffic, known attacks, and unknown traffic. We describe the process of using the AE algorithm [27], [28] used in CAIDS to identify known and unknown traffic, and the use of the CDDRF to identify normal and attack traffic in known traffic. The overall framework is shown in Fig. 2.

A. Unknown Flow

In traffic filtering of IoV, we first use the AE to learn the distribution of “normal traffic” within the feature space, which serves as our sole known reference. We input a sample set of normal traffic into the AE network, extracting and reconstructing a set of feature representations $X^t = \{X_1^t, X_2^t, X_3^t, \dots\}$ through the encoding-decoding process. When the entire traffic set $X = \{X_1, X_2, X_3, \dots\}$ is reconstructed by the same network, we obtain $X^r = \{X_1^r, X_2^r, X_3^r, \dots\}$. For each traffic record, we calculate the difference metric $\Phi = D(X_i, X_i^r)$ between the original and the reconstructed data, with commonly used indicators including the mean squared error (MSE). If the error Φ for a particular flow exceeds the threshold P , it is deemed to deviate significantly from the distribution of normal samples and may represent an unknown flow; otherwise, it is classified as known flow. We have set aside a verification set of normal flows that is independent of the training set. We first reconstruct this batch of actual normal flows using the same AE to obtain the corresponding error distribution $\{\Phi_j^{(val)}\}$. Subsequently, we apply statistical methods to determine the mean μ of the error distribution, adding several times the standard deviation σ (for example, $\mu + 3\sigma$) to establish the upper limit. In this manner, the threshold P is derived from the statistical analysis of the normal flow error, allowing for a more reliable identification of unknown flows. This process further aids in classifying known flows to ensure that various disguised attacks do not infiltrate the normal network.

To address threshold drift, we now maintain a rolling buffer of the most recent normal-traffic reconstruction errors and, at fixed intervals (for example, every hour or after collecting 500 new normal samples), recompute P so that it marks the upper bound of “normal” variation. As new errors arrive, we add them to the buffer and drop the oldest ones, then set P to the point below which 95% of these errors fall. This sliding-window update allows P to track gradual shifts in vehicle behavior or network conditions without overreacting to transient spikes.

B. Known Flow

In stage 2, CDDRF calls two classification sub-models a^r and b^r for each traffic sample, and each model outputs a category probability $\mathbf{a}^p$ and $\mathbf{b}^p$. We extract the maximum probability value from each of these two category probability, recorded as $\max(\mathbf{a}^p)$ and $\max(\mathbf{b}^p)$, and then compare them, marking the larger one as $\hat{Max}$. The category label corresponding to $\hat{Max}$ is the final decision, and the traffic is judged as “normal” or “known attack”. All traffic judged as abnormal will be discarded, and only normal traffic is allowed to pass.

To reveal the generation mechanism of $\{\mathbf{a}^r, \mathbf{b}^r\}$, Fig. 3 shows the entire processing flow. Assume that the traffic set to be

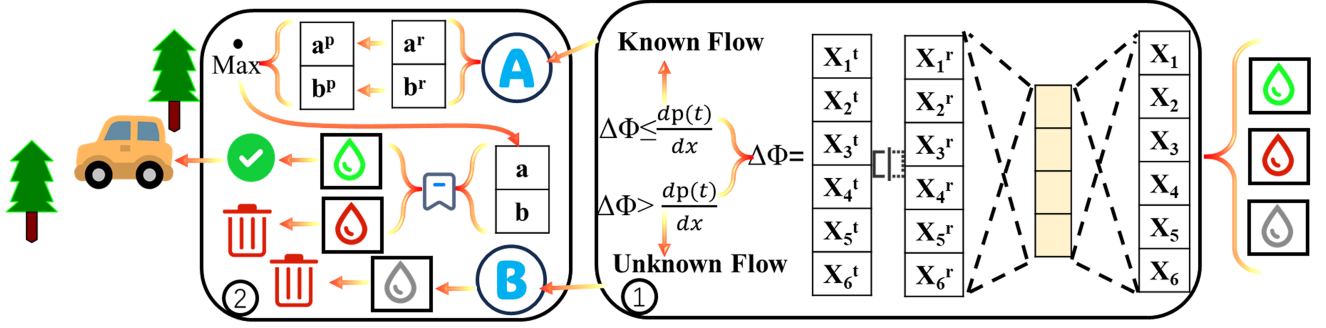


Fig. 2. CAIDS System Framework.

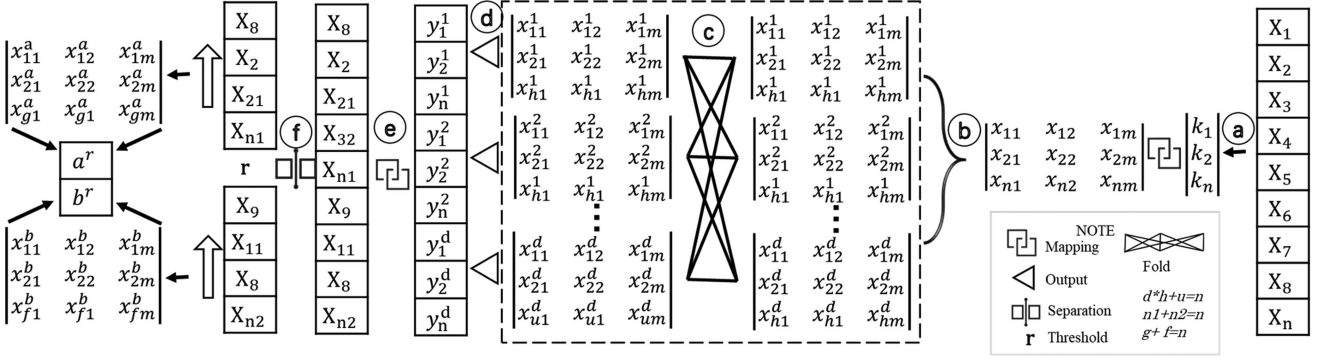


Fig. 3. CDDRF Framework.

detected is $\mathbf{X} = \{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_n\}$. In stage a, the K-fold cross-validation strategy is used to divide $\mathbf{X}$ into K subsets to ensure the consistency of the feature distribution of each subset with the overall data. In stage b, the random forest model is trained with K-1 subsets, and the remaining subset is used as the validation set. The trained model is used to classify it and calculate the category probability of each sample. In stage c, the validation set is switched cyclically until all subsets are predicted once, thereby obtaining a complete category probability prediction set. In stage d, these probability results are aggregated into a vector $\mathbf{y}_n^d$. And map them one by one with the corresponding samples $\mathbf{X}_n$. Then in stage e, sort the probability values in $\mathbf{y}_n^d$ from large to small, and adjust the order of the mapped samples synchronously to form a sorted sample sequence. Finally, in stage f, according to the preset threshold r , The samples with probability values greater than r in $\mathbf{y}_n^d$ are divided into high-confidence training subsets $\mathbf{a}^r$, and the samples with probability values greater than r are divided into low-confidence training subsets $\mathbf{b}^r$. Then, the random forest algorithm is used to train $\mathbf{a}^r$ and $\mathbf{b}^r$ independently, and finally a hierarchical model $\{\mathbf{a}^r, \mathbf{b}^r\}$ is obtained.

IV. CAIDS UNKNOWN MODEL

This section mainly establishes a mathematical model for the CAIDS model to identify known traffic and unknown traffic. This mechanism is called the unknown attack identification mechanism of the CAIDS model. The algorithm 1 shows the main framework of AE.

A. Data Definition and Merging

We begin with two datasets: one containing only normal (benign) samples, and one containing only abnormal samples. We define the benign dataset as:

$$D_{\text{normal}} = \{(\mathbf{x}_i, 0)\}_{i=1}^{N_0} \quad (1)$$

where $\mathbf{x}_i \in \mathbb{R}^d$ is the i -th feature vector (for instance, a network flow record, system log embedding, etc.), and the label 0 indicates “normal”. We define the abnormal dataset as:

$$D_{\text{anomaly}} = \{(\mathbf{x}_j, 1)\}_{j=1}^{N_1} \quad (2)$$

where $\mathbf{x}_j \in \mathbb{R}^d$ is the j -th feature vector, and the label 1 indicates “unknown abnormal”. Then, Combine these two datasets into a single dataset:

$$D = D_{\text{normal}} \cup D_{\text{anomaly}} = \{(\mathbf{x}_k, y_k)\}_{k=1}^{N_0+N_1} \quad (3)$$

With $y_k \in \{0, 1\}$. Here, $N = N_0 + N_1$ is the total number of samples.

B. Data Preprocessing

Before training the model, we typically conduct a series of data preprocessing steps:

1) *High-Cardinality Feature Removal*: If certain categorical features (e.g., IP addresses, host names, etc.) have extremely large cardinality, directly one-hot encoding them may lead to an enormous feature space. We remove or transform those features. Denote the set of high-cardinality columns by $\mathcal{C}_{\text{high}}$. We drop

Algorithm 1: AE Unknown Attack Identification.

Require: $D_{\text{normal}}, D_{\text{anomaly}}, d, \eta$
Ensure: Predicted labels $\hat{y}$ for D_{test}

- 1: Combine datasets: $D = D_{\text{normal}} \cup D_{\text{anomaly}}$
- 2: Remove high-cardinality features
- 3: One-hot encode categorical features
- 4: Impute missing values with means
- 5: $X_{\text{scaled}} \leftarrow \text{Standardize}(X)$
- 6: Compute $\Sigma = \frac{1}{N} X_{\text{scaled}}^\top X_{\text{scaled}}$
- 7: Eigen-decompose $\Sigma = V \Lambda V^\top$
- 8: Select k based on η
- 9: $X_{\text{pca}} \leftarrow X_{\text{scaled}} P_k$
- 10: $X_{\text{train}}, X_{\text{val}} \leftarrow \text{Split}(X_{\text{pca}}, y, \text{only normal})$
- 11: $X_{\text{test}}, y_{\text{test}} \leftarrow \text{PrepareTestSet}(D)$
- 12: Initialize θ
- 13: Optimize θ to minimize reconstruction loss on X_{train}
- 14: $\tau_{\text{mse}} \leftarrow \text{ComputePercentiles}(X_{\text{train}}, \theta, \eta)$
- 15: **for** each $\mathbf{x}_j \in X_{\text{test}}$ **do**
- 16: Compute $e_j, e'_j \leftarrow \text{ReconstructionErrors}(\mathbf{x}_j, \theta)$
- 17: **if** $e_j > \tau_{\text{mse}}$ **then**
- 18: $\hat{y}_j \leftarrow 1$
- 19: **else**
- 20: $\hat{y}_j \leftarrow 0$
- 21: **end if**
- 22: **end for**

return $\hat{y}$

them from D .

$$D \leftarrow D \setminus \mathcal{C}_{\text{high}} \quad (4)$$

2) *One-Hot Encoding*: Let $\mathcal{C}_{\text{cat}}$ be the remaining categorical features. For each categorical feature $c \in \mathcal{C}_{\text{cat}}$ with m categories, we map it to an m -dimensional 0/1 vector (one-hot encoding).

$$\mathbf{c} \mapsto (0, \dots, 0, 1, 0, \dots, 0) \in \mathbb{R}^m \quad (5)$$

This results in an expanded feature dimension. After this step, all remaining features are numerical.

3) *Missing Value Imputation*: Any missing values in the numerical features can be imputed by their column mean. We replace each missing entry with:

$$\hat{x} = \frac{1}{\text{count of non-missing}} \sum_{\text{non-missing}} x \quad (6)$$

4) *Final Feature Matrix*: Eventually, we form a numerical matrix and label vector. Where d' is the new dimensionality after removing columns and performing one-hot encoding.

$$X \in \mathbb{R}^{N \times d'}, \mathbf{y} \in \{0, 1\}^N \quad (7)$$

C. Feature Standardization

To help certain models converge more easily, we standardize each feature to zero mean and unit variance.

$$x_{\text{scaled}, ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (8)$$

where μ_j and σ_j are the mean and standard deviation of the j -th feature across the dataset. Finally, we obtain $X_{\text{scaled}} \in \mathbb{R}^{N \times d'}$.

D. PCA (Principal Component Analysis) Dimensionality Reduction

Given the standardized data $X_{\text{scaled}} \in \mathbb{R}^{N \times d'}$, compute via (9) and Eigen-Decomposition via (10).

$$\Sigma = \frac{1}{N} X_{\text{scaled}}^\top X_{\text{scaled}} \in \mathbb{R}^{d' \times d'} \quad (9)$$

$$\Sigma = V \Lambda V^\top \quad (10)$$

where $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_{d'})$ with $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$, and V contains the corresponding eigenvectors. Then, choose the smallest k such that

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^{d'} \lambda_i} \geq \text{Desired Ratio} \quad (11)$$

Let $P_k = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k]$. Project the data onto these k principal components.

$$X_{\text{pca}} = X_{\text{scaled}} P_k \in \mathbb{R}^{N \times k} \quad (12)$$

E. Dataset Processing

Since we want the AE to learn only normal behavior, we allocate *only normal samples* to training and validation.

$$\Omega_0 = \{i : y_i = 0\}, \quad \Omega_1 = \{i : y_i = 1\} \quad (13)$$

We randomly split Ω_0 into two disjoint sets for training and validation, Both contain only normal data (14). For evaluation, we use test set have Normal and Abnormal (15).

$$X_{\text{train}} = X_{\text{pca}}(\Omega_{0,\text{train}}), \quad X_{\text{val}} = X_{\text{pca}}(\Omega_{0,\text{val}}) \quad (14)$$

$$X_{\text{test}} = X_{\text{pca}}(\Omega_0 \cup \Omega_1), \quad y_{\text{test}} = \mathbf{y}(\Omega_0 \cup \Omega_1) \quad (15)$$

F. AE Construction Training

An AE is a neural network aiming to learn an identity mapping.

$$f_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^k \quad \text{with } \theta = (\phi, \psi) \quad (16)$$

$$f_\theta(\mathbf{x}) = D_\psi(E_\phi(\mathbf{x})) \quad (17)$$

- $E_\phi(\mathbf{x})$: **Encoder**, transforms input $\mathbf{x} \in \mathbb{R}^k$ to some hidden representation $\mathbf{h}$.
- $D_\psi(\mathbf{h})$: **Decoder**, maps $\mathbf{h}$ back to the same dimension as $\mathbf{x}$.

For example, one might have hidden layers of size 64, 32, 16 in the encoder, then mirror them (16, 32, 64) in the decoder, culminating in a k -dimensional output. We train only on normal data, aiming to minimize the discrepancy between $\mathbf{x}_i$ and its reconstruction $f_\theta(\mathbf{x}_i)$. A common choice is MSE:

$$\mathcal{L}(\theta) = \sum_{i \in \Omega_{\text{train}}} \|\mathbf{x}_i - f_\theta(\mathbf{x}_i)\|^2 \quad (18)$$

where $\Omega_{\text{train}} \subset \Omega_0$ denotes the indices of normal samples in the training set. We iteratively update θ (weights and biases) via

an optimizer like Adam or SGD until we reach a maximum number of epochs, or the validation loss no longer improves after a certain threshold. After the AE is trained on normal samples, we derive an error threshold from the training set. Then, we compute reconstruction errors on the training set. For each $\mathbf{x}_i \in X_{\text{train}}$.

$$e_i = \|\mathbf{x}_i - f_\theta(\mathbf{x}_i)\|^2 \quad (19)$$

Let τ_{mse} be the η percentile of $\{e_i\}$. Any sample whose error exceeds these thresholds can be flagged as abnormal. We then use these thresholds on the test set $(X_{\text{test}}, y_{\text{test}})$. Testing with MSE, For each test sample $\mathbf{x}_j$, compute $e_j = \|\mathbf{x}_j - f_\theta(\mathbf{x}_j)\|^2$. Then, make predictions. Here, $\hat{y}_j = 1$ indicates an abnormal.

$$\hat{y}_j^{(\text{mse})} = \begin{cases} 1, & e_j > \tau_{\text{mse}} \\ 0, & \text{otherwise} \end{cases} \quad (20)$$

V. CAIDS KNOWN MODELS

This section primarily establishes the mathematical model for CAIDS, utilized to distinguish normal traffic from known attack traffic. This approach is referred to as the Known Traffic Identification Mechanism within CAIDS.

The stratified K-fold DRF method is built on the traditional stratified K-fold cross-validation. For each validation set in each fold, an initial random forest model is trained to calculate the highest predicted probability of the known category for each sample, denoted as α_i . Subsequently, a confidence threshold r is set to classify samples with $\alpha_i \geq r$ into the “high-confidence” dataset and samples with $\alpha_i < r$ into the “low-confidence” dataset. Then, dedicated random forest classifiers are trained for these high-confidence and low-confidence subsets respectively. High-confidence datasets refer to samples whose predicted probability of the random forest in the known category of the sample is higher than the threshold r , and high-confidence datasets refer to samples whose predicted probability of the random forest in the known category of the sample is lower than the threshold r . In the test phase, each sub-model provides the predicted category and its predicted probability for each test sample. Finally, the sub-model with the highest predicted probability is selected to generate the classification result. This method ensures high accuracy for easy-to-classify samples while enhancing robustness for difficult-to-classify or noisy samples. Algorithm 2 illustrates the primary framework of the CDDRF method. Table I summarizes the variables and formula symbols.

We define the training set and test set. Each sample $\mathbf{x}_i \in \mathbb{R}^d$ is a d -dimensional feature vector and y_i is its class label (e.g. an attack type):

$$D_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N_{\text{train}}}, \quad D_{\text{test}} = \{(\mathbf{x}_j, y_j)\}_{j=1}^{N_{\text{test}}} \quad (21)$$

We arrange these into a feature matrix and label vector for each set:

$$X_{\text{train}} \in \mathbb{R}^{N_{\text{train}} \times d}, \quad \mathbf{y}_{\text{train}} \in \{\text{Attack}_1, \dots, \text{Attack}_C\}^{N_{\text{train}}} \quad (22)$$

$$X_{\text{test}} \in \mathbb{R}^{N_{\text{test}} \times d}, \quad \mathbf{y}_{\text{test}} \in \{\text{Attack}_1, \dots, \text{Attack}_C\}^{N_{\text{test}}} \quad (23)$$

TABLE I
KEY SYMBOL AND DEFINITION

Symbol	Definition
$D_{\text{normal}}, D_{\text{anomaly}}$	Benign / abnormal sets
N_0, N_1	Sizes of $D_{\text{normal}}, D_{\text{anomaly}}$
$D = D_{\text{normal}} \cup D_{\text{anomaly}}$	Combined dataset
N	Total number of samples
$\mathbf{x}_i \in \mathbb{R}^d$	Feature vector
y_i	Label (0: normal, 1: anomaly)
d, d'	Original / post-encoding dimensions
X_{scaled}	Standardized feature matrix
X_{pca}	Data after PCA projection
P_k	Projection onto top- k principal components
f_θ	Autoencoder mapping $D_\psi \circ E_\phi$
e_i	Reconstruction errors (MSE)
η	Percentile for threshold selection
τ_{mse}	Error thresholds at η -percentile
$D_{\text{train}}, D_{\text{test}}$	Train / test sets (known attacks)
K	Number of CV folds
r	Confidence threshold for splitting
$\mathcal{I}_o^{(k)}, \mathcal{I}_b^{(k)}$	High / low-confidence indices in fold k
RF_{full}	Fallback random forest on all train data
$\hat{y}$	Predicted label

To preserve class proportions in each fold, we apply stratified K -fold cross-validation on the training set. Let $\{(\mathcal{I}_{\text{train}}^{(k)}, \mathcal{I}_{\text{val}}^{(k)})\}_{k=1}^K$ be a partition of the index set $\{1, \dots, N_{\text{train}}\}$ into K disjoint training/validation index pairs, satisfying

$$\mathcal{I}_{\text{train}}^{(k)} \cup \mathcal{I}_{\text{val}}^{(k)} = \{1, \dots, N_{\text{train}}\}, \quad \mathcal{I}_{\text{train}}^{(k)} \cap \mathcal{I}_{\text{val}}^{(k)} = \emptyset \quad (24)$$

And maintaining the overall label distribution in each fold. For each fold k , we extract the corresponding training and validation subsets by indexing into X_{train} and $\mathbf{y}_{\text{train}}$:

$$X_{\text{train}}^{(k)} = X_{\text{train}}[\mathcal{I}_{\text{train}}^{(k)}, :], \quad \mathbf{y}_{\text{train}}^{(k)} = \mathbf{y}_{\text{train}}[\mathcal{I}_{\text{train}}^{(k)}] \quad (25)$$

$$X_{\text{val}}^{(k)} = X_{\text{train}}[\mathcal{I}_{\text{val}}^{(k)}, :], \quad \mathbf{y}_{\text{val}}^{(k)} = \mathbf{y}_{\text{train}}[\mathcal{I}_{\text{val}}^{(k)}] \quad (26)$$

This yields K pairs $(X_{\text{train}}^{(k)}, \mathbf{y}_{\text{train}}^{(k)})$ and $(X_{\text{val}}^{(k)}, \mathbf{y}_{\text{val}}^{(k)})$, each preserving the original class distribution for robust model evaluation. In fold k , we first train an “initial” random forest classifier, where $\mathcal{L}$ represents the loss function for training a random forest.

$$\text{RF}_{\text{initial}}^{(k)} = \arg \min_{\theta} [\mathcal{L}(\theta; X_{\text{train}}^{(k)}, \mathbf{y}_{\text{train}}^{(k)})] \quad (27)$$

For each fold k , let,

$$\mathbf{p}_i^{(k)} = \text{RF}_{\text{initial}}^{(k)}.\text{predict_proba}(\mathbf{x}_i) \in \mathbb{R}^C, \quad i \in \mathcal{I}_{\text{val}}^{(k)} \quad (28)$$

Here $\mathbf{p}_i^{(k)} = [p_{i,1}^{(k)}, \dots, p_{i,C}^{(k)}]^\top$ is the vector of predicted probabilities for the C classes. We then define the confidence score for each sample as the maximum predicted probability:

$$\alpha_i^{(k)} = \max_{c=1, \dots, C} p_{i,c}^{(k)} \in [0, 1] \quad (29)$$

According to the preset threshold r , samples with probability values greater than r in α are divided into high-confidence training subsets $\mathbf{a}^r$, and samples with probability values greater than r are divided into low-confidence training subsets $\mathbf{b}^r$.

$$\mathcal{I}_a^{(k)} = \{i \in \mathcal{I}_{\text{val}}^{(k)} : \alpha_i^{(k)} \geq r\} \quad (30)$$

$$\mathcal{I}_b^{(k)} = \{i \in \mathcal{I}_{\text{val}}^{(k)} : \alpha_i^{(k)} < r\} \quad (31)$$

Algorithm 2: CDDRF Known Traffic Identification.

Require: $D_{\text{train}}, D_{\text{test}}, d, C, K, r$
Ensure: Predicted labels $\hat{y}$ for D_{test}

- 1: Organize D_{train} and D_{test} into $X, \mathbf{y}$ and $X_{\text{test}}, \mathbf{y}_{\text{test}}$
- 2: Perform stratified K -fold CV to obtain folds
- 3: **for** $k = 1$ to K **do**
- 4: Train $\text{RF}_{\text{initial}}^{(k)} \leftarrow \text{TrainRF}(X_{\text{train}}^{(k)}, \mathbf{y}_{\text{train}}^{(k)})$
- 5: Compute $\alpha_i \leftarrow \max(\text{RF}_{\text{initial}}^{(k)}.\text{predict_proba}(\mathbf{x}_i))$ for all $i \in \mathcal{I}_{\text{val}}^{(k)}$
- 6: Split into $\mathcal{I}_a^{(k)} \leftarrow \{i \mid \alpha_i \geq r\}$ and $\mathcal{I}_b^{(k)} \leftarrow \{i \mid \alpha_i < r\}$
- 7: **if** $\mathcal{I}_a^{(k)} \neq \emptyset$ **then**
- 8: Train $\text{RF}_a^{(k)} \leftarrow \text{TrainRF}(X[\mathcal{I}_a^{(k)}], \mathbf{y}[\mathcal{I}_a^{(k)}])$
- 9: **end if**
- 10: **if** $\mathcal{I}_b^{(k)} \neq \emptyset$ **then**
- 11: Train $\text{RF}_b^{(k)} \leftarrow \text{TrainRF}(X[\mathcal{I}_b^{(k)}], \mathbf{y}[\mathcal{I}_b^{(k)}])$
- 12: **end if**
- 13: **end for**
- 14: Train fallback $\text{RF}_{\text{full}} \leftarrow \text{TrainRF}(X, \mathbf{y})$
- 15: **for each** $\mathbf{x}_* \in X_{\text{test}}$ **do**
- 16: Initialize $\alpha_a^{\max} \leftarrow 0, \hat{y}_a \leftarrow \text{None}$
- 17: Initialize $\alpha_b^{\max} \leftarrow 0, \hat{y}_b \leftarrow \text{None}$
- 18: **for** $k = 1$ to K **do**
- 19: **if** trained $\text{RF}_a^{(k)}$ **then**
- 20: $\mathbf{p}_a \leftarrow \text{RF}_a^{(k)}.\text{predict_proba}(\mathbf{x}_*)$
- 21: Update $\alpha_a^{\max}, \hat{y}_a \leftarrow \max_prob(\mathbf{p}_a)$ if higher
- 22: **end if**
- 23: **if** trained $\text{RF}_b^{(k)}$ **then**
- 24: $\mathbf{p}_b \leftarrow \text{RF}_b^{(k)}.\text{predict_proba}(\mathbf{x}_*)$
- 25: Update $\alpha_b^{\max}, \hat{y}_b \leftarrow \max_prob(\mathbf{p}_b)$ if higher
- 26: **end if**
- 27: **end for**
- 28: **if** $\alpha_a^{\max} \geq \alpha_b^{\max}$ and $\alpha_a^{\max} > 0$ **then**
- 29: $\hat{y}_* \leftarrow \hat{y}_a$
- 30: **else if** $\alpha_b^{\max} > \alpha_a^{\max}$ and $\alpha_b^{\max} > 0$ **then**
- 31: $\hat{y}_* \leftarrow \hat{y}_b$
- 32: **else**
- 33: $\hat{y}_* \leftarrow \text{RF}_{\text{full}}.\text{predict}(\mathbf{x}_*)$
- 34: **end if**
- 35: Save $\hat{y}_*$
- 36: **end for**
- return** $\hat{y}$

The set $\mathcal{I}_a^{(k)}$ contains all validation samples whose highest predicted class probability meets or exceeds r (high-confidence), while $\mathcal{I}_b^{(k)}$ contains those below the threshold (low-confidence). We define:

$$X_a^{(k)} = X_{\text{val}}^{(k)}(\mathcal{I}_a^{(k)}), \quad \mathbf{y}_a^{(k)} = \mathbf{y}_{\text{val}}^{(k)}(\mathcal{I}_a^{(k)}) \quad (32)$$

$$X_b^{(k)} = X_{\text{val}}^{(k)}(\mathcal{I}_b^{(k)}), \quad \mathbf{y}_b^{(k)} = \mathbf{y}_{\text{val}}^{(k)}(\mathcal{I}_b^{(k)}) \quad (33)$$

Here, subset a contains samples that the initial model predicts with high confidence ($\alpha \geq r$), while subset b contains low-confidence samples. We train two additional random forest

classifiers if the subsets are non-empty:

$$\text{RF}_a^{(k)} = \arg \min_{\theta} \left[\mathcal{L}(\theta; X_a^{(k)}, \mathbf{y}_a^{(k)}) \right] \quad (34)$$

$$\text{RF}_b^{(k)} = \arg \min_{\theta} \left[\mathcal{L}(\theta; X_b^{(k)}, \mathbf{y}_b^{(k)}) \right] \quad (35)$$

These models specialize in classifying either high-confidence (a) or low-confidence (b) samples from the validation set. This can effectively solve the problem of low-confidence data affecting high-confidence data.

For each test sample $\mathbf{x}_*$ in D_{test} .

$$\hat{\mathbf{p}}_{*,a}^{(k)} = \text{RF}_a^{(k)}.\text{predict_proba}(\mathbf{x}_*), \quad \hat{\alpha}_{*,a}^{(k)} = \max(\hat{\mathbf{p}}_{*,a}^{(k)}) \quad (36)$$

$$\hat{\mathbf{p}}_{*,b}^{(k)} = \text{RF}_b^{(k)}.\text{predict_proba}(\mathbf{x}_*), \quad \hat{\alpha}_{*,b}^{(k)} = \max(\hat{\mathbf{p}}_{*,b}^{(k)}) \quad (37)$$

We get corresponding class predictions $\hat{y}_{*,a}^{(k)}$ or $\hat{y}_{*,b}^{(k)}$ by choosing the highest-probability class. If either model does not exist (i.e., if $X_a^{(k)}$ or $X_b^{(k)}$ was empty), we set the respective probability to 0 or mark it as invalid.

After all K folds, for each test sample i , we collect:

$$\hat{\alpha}_{i,a}^{(1)}, \dots, \hat{\alpha}_{i,a}^{(K)}, \quad \hat{\alpha}_{i,b}^{(1)}, \dots, \hat{\alpha}_{i,b}^{(K)} \quad (38)$$

One way to form the final prediction $\hat{y}_i$ is:

- 1) Let $\alpha_{i,a}^{\max} = \max_{k: \text{valid } \hat{\alpha}_{i,a}^{(k)}} \hat{\alpha}_{i,a}^{(k)}$, and similarly $\alpha_{i,b}^{\max} = \max_{k: \text{valid } \hat{\alpha}_{i,b}^{(k)}} \hat{\alpha}_{i,b}^{(k)}$.
- 2) Identify which model yields the higher maximum probability:

$$\hat{y}_i = \begin{cases} \hat{y}_{i,a}^{(\text{best})}, & \text{if } \alpha_{i,a}^{\max} \geq \alpha_{i,b}^{\max}, \\ \hat{y}_{i,b}^{(\text{best})}, & \text{otherwise.} \end{cases}$$

- 3) If both $\alpha_{i,a}^{\max}$ and $\alpha_{i,b}^{\max}$ are 0 (no valid model), fall back to an initial random forest trained on the entire training set, denoted $\text{RF}_{\text{initial,full}}$.

In each fold k , we compute metrics such as accuracy on the validation subset. Similarly compute Precision, Recall, F1, etc.

$$\text{Accuracy}^{(k)} = \frac{1}{|\mathcal{I}_{\text{val}}^{(k)}|} \sum_{i \in \mathcal{I}_{\text{val}}^{(k)}} \mathbf{1} \{ \hat{y}_i^{(k)} = y_i \} \quad (39)$$

Average the metrics over all folds:

$$\overline{\text{Accuracy}} = \frac{1}{K} \sum_{k=1}^K \text{Accuracy}^{(k)}, \quad \overline{\text{F1}} = \frac{1}{K} \sum_{k=1}^K \text{F1}^{(k)}, \dots \quad (40)$$

VI. ANALYSIS

This section presents an analytical evaluation of CAIDS's ability to identify known, and unknown traffic.

A. Known Flow Analysis

To comprehensively evaluate the performance of the CDDRF model, we conducted comparative experiments on four benchmark datasets commonly used in IoV and edge environments: EDGE-IIoT, TON-IoT, CICIDS2017, and UNSW-NB15. These datasets can approximate real IoV scenarios well in terms of

TABLE II
COMPARED ALGORITHMS' PARAMETER SETTINGS

Algorithm	Parameter Settings
CDDRF	n_estimators=100, random_state=42, k=5
Logistic Regression	max_iter=2000, multi_class='multinomial', solver='lbfgs'
Random Forest	n_estimators=100, random_state=42
XGBoost	max_depth=10, n_estimators=500, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, gamma=0, reg_alpha=0.01, reg_lambda=1.0
Deep LSTM	layers=[128, 64, 32], dropout=0.2, recurrent_dropout=0.2, optimizer=Adam(lr=0.001), epochs=50, batch_size=64
Deep 1D CNN	filters=[128, 64, 32], kernel_size=3, dropout=[0.2, 0.2, 0.3], optimizer=Adam(lr=0.001), epochs=50, batch_size=64
Deep Transformer	blocks=2, heads=4, key_dim=64, dropout=0.1, optimizer=Adam(lr=0.001), epochs=50, batch_size=64

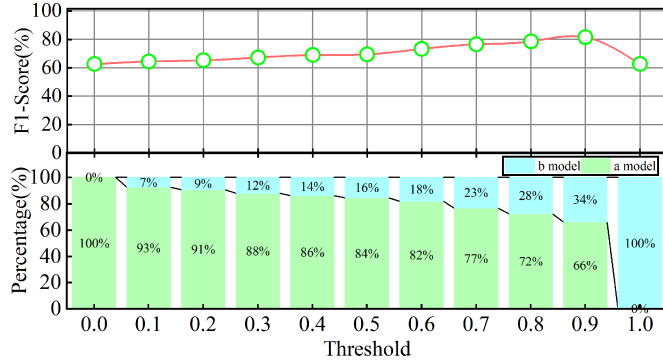


Fig. 4. Threshold impact on CDDRF F1-score and a/b sample ratio.

protocol diversity (including TCP/UDP), high dynamics and mobility of network topology, and a variety of attack types [17], [20], [29]. We employed Logistic Regression (LR), Random Forest (RF), XGBoost, LSTM, 1D CNN, and Transformer models as comparative benchmarks.

To maximize the F1-score on the validation set, we utilized the same validation set for both the CDDRF model and each baseline model, to maximize the validation set F1-score, and determine the optimal configuration through manual traversal. Random seeds, layers, rounds, and learning rates were uniformly applied during the training of deep models, with the parameter configuration detailed in Table II. All datasets were divided into training and test sets in a 70:30 ratio. Preprocessing steps included the removal of high-cardinality features, imputation of missing values using the mean, one-hot encoding of categorical features, and PCA dimensionality reduction to improve feature expression quality and reduce redundancy.

We first systematically scanned different candidate thresholds (from 0.0 to 1.0, with a step size of 0.1) on another independent UNSW-NB15 validation set, recorded the F1-score at each threshold, and selected the threshold r with the best or most robust F1-score. For the convenience of comparison, we merged the two sets of data and plotted them as shown in Fig. 4. The experimental results show that as the threshold increases, the F1-score shows a clear upward trend, especially when the threshold reaches 0.8, the F1-score increases from 78.69% to 82.52%. This shows that the increase in the sample ratio of the a/b subset will lead to a significant difference in the confidence between the a/b subsets. However, when the threshold continues to increase, the F1-score decreases instead, approaching the performance of a single random forest. This phenomenon shows that when the sample ratio of the a/b subset increases to a

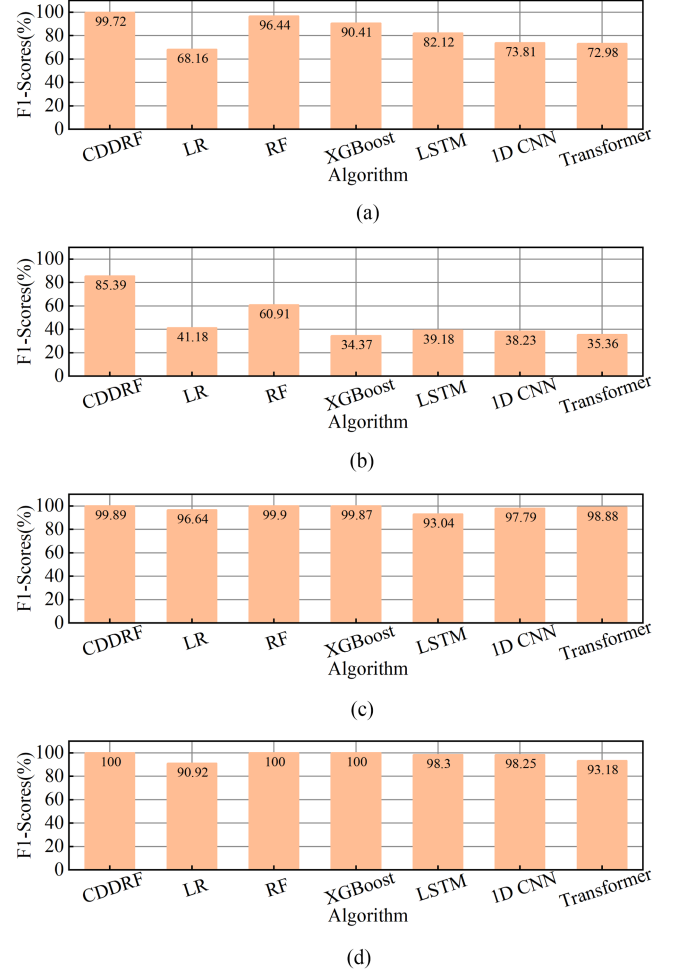


Fig. 5. F1-scores comparison of CDDRF with other algorithms. (a) EDGE-IIoT dataset. (b) UNSW-NB15 dataset. (c) CICIDS2017 dataset. (d) TON-IoT dataset.

certain extent, appropriately dividing the datasets with different confidence levels for training can help improve the model's robustness to easily distinguishable data, while increasing the focus on difficult-to-distinguish data.

Fig. 5(a) presents the F1-score comparison between CDDRF and six benchmark methods. CDDRF achieves an F1-score of 99.72%, outperforming Random Forest (96.44%) and XGBoost (90.41%), which demonstrates its superior ability to capture complex and nonlinear traffic patterns. Among the deep learning baselines, LSTM attains 82.12%, 1D CNN 73.81%, and Transformer 72.98%. Their moderate performance under identical optimization settings suggests that these architectures may be less well matched to the specific feature distributions of our datasets. Logistic regression reaches only 68.16%, underscoring the significant nonlinear relationships present in the intrusion detection task. Fig. 5(c) and (d) demonstrate that CDDRF consistently outperforms both traditional and deep learning baselines in the CICIDS2017 and TON-IoT datasets. In CICIDS2017, the CDDRF F1-score exceeds that of logistic regression by 3.25% and that of LSTM by 6.85%; In TON-IoT, it outperforms logistic regression by 9.18% and LSTM by 1.70%. In contrast, Fig. 5(b)

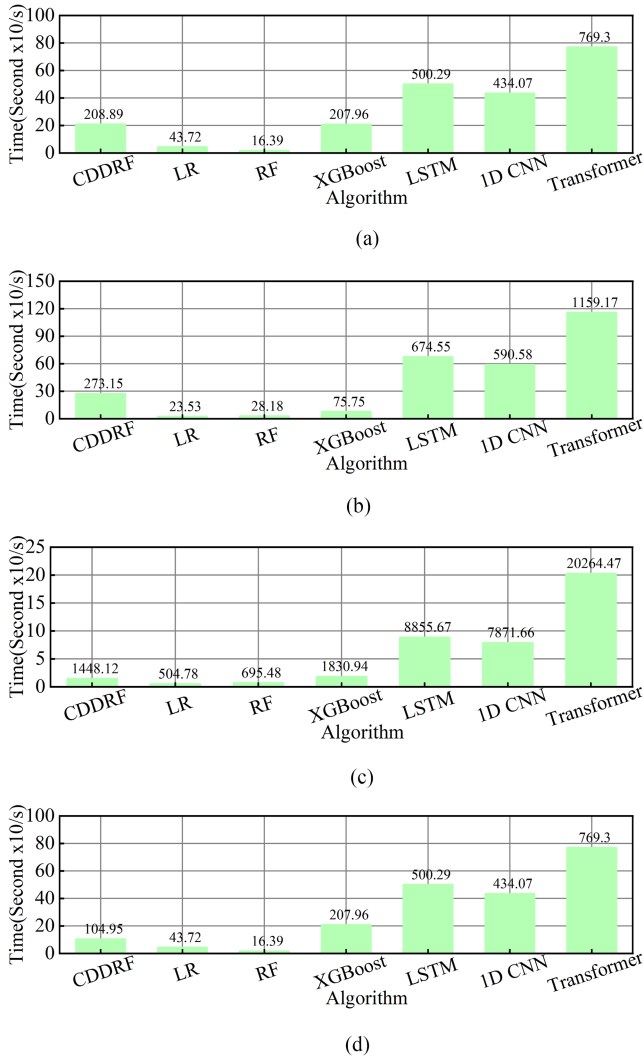


Fig. 6. Time comparison of CDDRF with other algorithms. (a) EDGE-IIoT dataset. (b) UNSW-NB15 dataset. (c) CICIDS2017 dataset. (d) TON-IIoT dataset.

shows that in UNSW-NB15 the nearest competitor, random forest, reaches only 60.91%, while deep models perform poorly, likely due to feature redundancy and class imbalance that impairs effective representation learning.

As shown in Fig. 6, the training time of CDDRF on the same dataset is slightly longer than that of traditional machine learning models (LR, RF, XGBoost), but their F1-score is significantly better than these methods. At the same time, compared with mainstream deep learning models (LSTM, 1D CNN, Transformer), CDDRF takes less time to train. CDDRF not only surpasses traditional algorithms in prediction performance but also surpasses deep models in efficiency, achieving the best balance between accuracy and time cost. In other datasets or network conditions, the optimal range of r may need to be re-evaluated. However, other experiments currently use a static threshold of $r = 0.9$, which can achieve better detection performance on all datasets under the current threshold, indicating that a single static threshold is sufficient to cope with different network

environments. In actual deployments, threshold re-evaluation measures can be used to adapt to traffic changes and threshold drift during deployment.

B. Unknown Flow Analysis

To evaluate unknown traffic detection, we use the TON-IIoT network traffic dataset. Normal traffic is randomly split 70%/30% into training and test sets to prevent data leakage and ensure fair evaluation. We train the AE solely on normal traffic, then evaluate its generalization by treating selected attack types from TON-IIoT as unknown traffic. Here, “unknown traffic” refers to a typical network attack type and its diverse combination that have never been seen in the training process, which belongs to a new attack type.

For hyperparameter tuning, we form a validation set with DoS, Backdoor, and Scanning attacks and optimize via grid search. To simulate zero-day conditions, the remaining ten attacks: DDoS, Injection, Password, XSS, Ransomware, MITM, Shellcode, Worms, Reconnaissance, Exploits, are treated as unknown and only appear in the open-set evaluation.

We compare CAIDS against eight IDS baselines (AUIDS, NIDS, LIDS, TLOC, SupEnML, IoT-PRIDS, ENCSA-PSO, HyperVision, MLMCS); see Section “Related Work” for details. The AE reconstruction-error threshold is set to 95%, as commonly adopted in prior studies [30], [31].

Fig. 7 shows how the training loss and validation loss of the AE change with the number of iterations under different unknown attack complexities. Here, we define “complexity” as the number of unknown attack categories in the test set: the more categories, the higher the complexity. Although all experiments used the same training set for model training, due to the different types of unknown attacks contained in different test sets, the preprocessing steps (including PCA dimensionality reduction and feature encoding) were performed independently, resulting in slightly different feature dimensions and encodings of the model input. It can be seen in the figure that as the complexity of the unknown attack increases, the validation loss rises and the gap between the training and validation curves gradually widens, which shows that the AE’s ability to fit and identify multi-category, high-complexity unknown attacks has significantly decreased.

Fig. 8 illustrates the impact of unknown traffic with varying complexity on the AE’s performance. Taking the xy -axis as an example, the classification outcomes are defined as follows:

- 00 represents true positive.
- 01 represents false positive.
- 10 represents false negative.
- 11 represents true negative.

According to Fig. 8, when the unknown-attack complexity was 1, the AE produced a false-positive rate of just 0.50% for benign traffic and a false-negative rate of 0.58% for attack traffic, demonstrating strong generalization to novel flows. Even at a complexity level of 3, the false-positive rate rose only slightly to 0.92%, while the false-negative rate remained at 0.58%, indicating the model still performed robustly. Beyond this point, the AE continued to detect unknown attacks accurately, but the

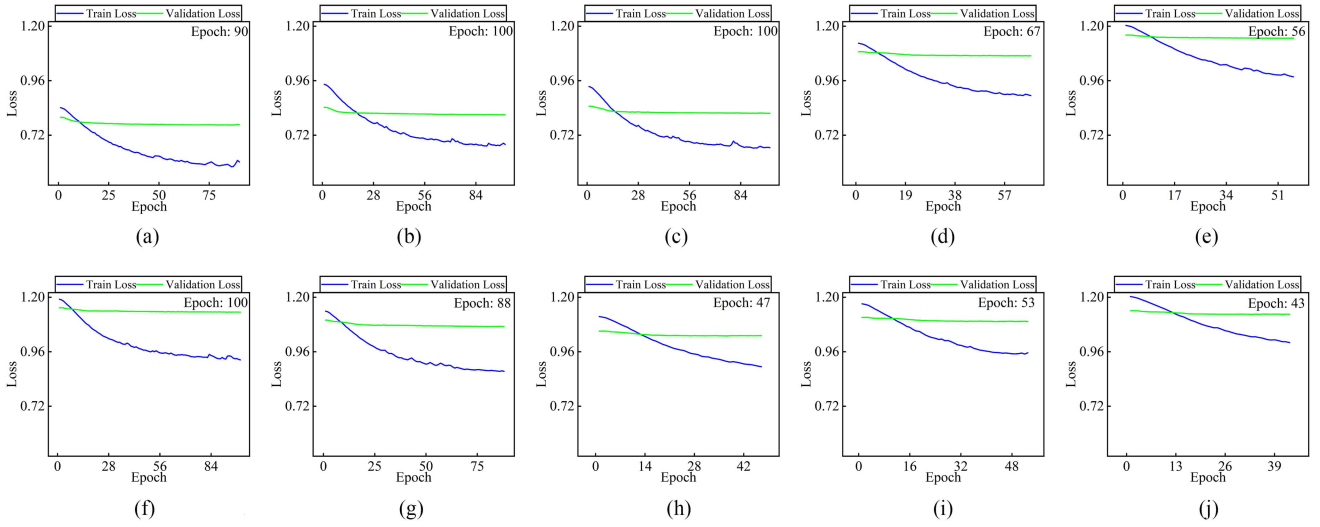


Fig. 7. Training loss and validation loss when training with known traffic. (a) Single class. (b) Double class. (c) Three class. (d) Four class. (e) Five class. (f) Six class. (g) Seven class. (h) Eight class. (i) Nine class. (j) Ten class.

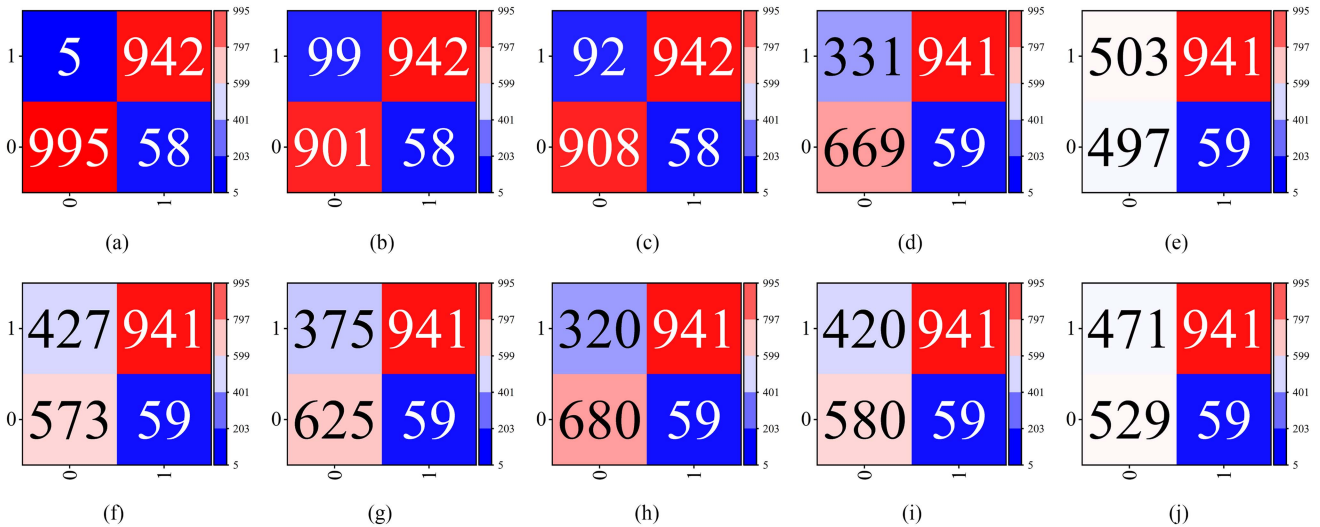


Fig. 8. Confusion matrix for identifying unknown traffic. (a) Single class. (b) Double class. (c) Three class. (d) Four class. (e) Five class. (f) Six class. (g) Seven class. (h) Eight class. (i) Nine class. (j) Ten class.

false-positive rate increased because higher-complexity attacks share more features with benign traffic, narrowing the margin that separates normal from malicious behavior and making misclassifications more likely.

Next, we calculated the anomaly recognition accuracy under all complexities based on the confusion matrix data. As shown in Fig. 10(a), when the complexity is single-class, its accuracy reaches 97%, which is enough to deal with various threats in the real environment. When the complexity does not exceed 3, the AE has sufficient accuracy to identify unknown traffic.

To find out the robustness of CAIDS in identifying unknown traffic, we calculated the system robustness of AUC (Area Under the Receiver Operating Characteristic Curve) under unknown attacks of all complexities. As shown in Fig. 9, when the

complexity of the unknown attack does not exceed 3, the ROC (Receiver Operating Characteristic) curve generated by CAIDS in the AE stage is always above the diagonal and close to the upper left corner, and the corresponding AUC is more than 0.95, up to 0.99. This shows that the system can achieve a high true positive rate (TPR) at an extremely low false positive rate (FPR), and the overall performance is excellent. As the number of unknown attack types increases to more than 3 (complexity > 3), the distribution of reconstruction errors caused by the diversified attack samples shifts, making it difficult for the original fixed threshold strategy to simultaneously take into account low false positives and high hits, which leads to a significant decrease in the AUC value and the position of the ROC curve. The above results verify the reliability of CAIDS in low-complexity unknown attacks.

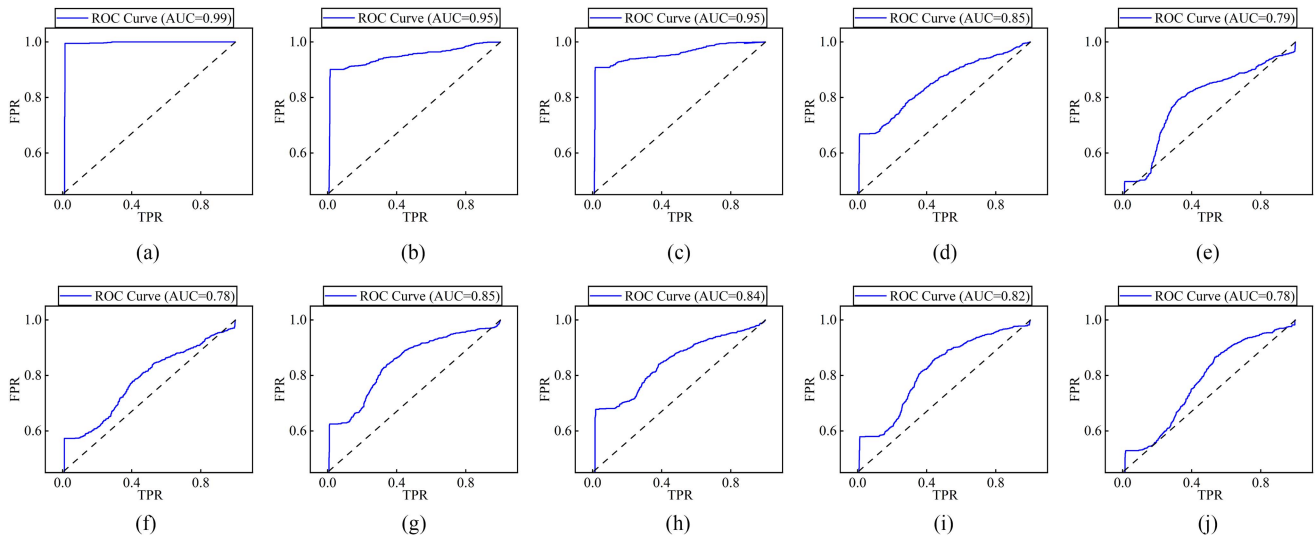


Fig. 9. AUC for identifying unknown traffic. (a) Single class. (b) Double class. (c) Three class. (d) Four class. (e) Five class. (f) Six class. (g) Seven class. (h) Eight class. (i) Nine class. (j) Ten class.

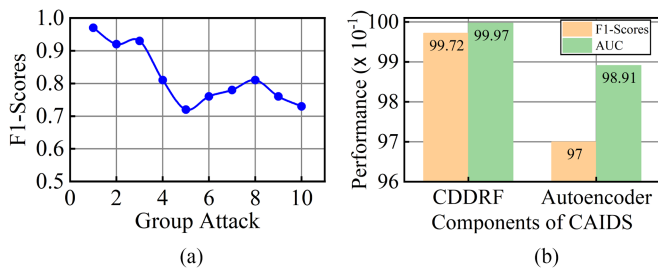


Fig. 10. Average performance of CAIDS. (a) F1-scores. (b) Performance analysis.

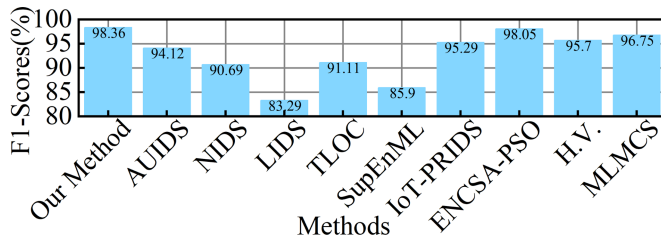


Fig. 11. F1-score comparison of CAIDS with other methods.

Since similar studies in recent years are all set in low-complexity scenarios, we combined the F1-scores and AUCs of CDDRF and AE under single-class complexity, as shown in Fig. 10(b). CAIDS has two components. CDDRF has an excellent F1-score of 99.72% and an AUC of 0.9997, while AE has an F1-score of 97.00% and an AUC of 0.9891. To compare the IDSs of the same single complexity, we calculated the average F1-score of CAIDS. As shown in Fig. 11, we combined the performance of CAIDS on known attacks and unknown attacks. Compared with recent similar studies, the comprehensive F1-score of CAIDS for known and unknown attacks is much higher than other studies, reaching 98.36%. Most similar studies only focus on a single aspect, while CAIDS

has made great improvements in both performance and actual performance.

VII. CONCLUSION

With the increasing sophistication and connectivity of modern vehicular networks, the challenge of effectively detecting and mitigating both known and unknown attacks remains a critical research focus. IDSs must contend with diverse threats, ranging from novel exploits that target previously unknown vulnerabilities to variations of established attack patterns. Addressing these challenges, we have proposed CAIDS, an integrated IDS that combines an AE module for detecting unknown traffic with a novel CDDRF model for identifying known intrusions. By actively updating the AE with filtered normal traffic, CAIDS maintains adaptability against emerging threats, thereby enhancing the detection accuracy for unknown attacks. Additionally, our CDDRF model uses hierarchical k-folding to separate high-confidence and low-confidence datasets, producing two complementary random forest classifiers that work in tandem for reliable, high-precision detection. Experimental comparisons with current state-of-the-art approaches confirm that CAIDS offers superior performance and flexibility under real-world conditions. Future research directions will focus on optimizing the system's real-time response, reducing resource overhead, and further strengthening model robustness against sophisticated adversarial strategies.

REFERENCES

- [1] J. Contreras-Castillo, S. Zeadally, and J. A. Guerrero-Ibañez, "Internet of Vehicles: Architecture, protocols, and security," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3701–3709, Oct. 2018.
- [2] S. Sharma and B. Kaushik, "A survey on Internet of Vehicles: Applications, security issues & solutions," *Veh. Commun.*, vol. 20, 2019, Art. no. 100182.
- [3] W. Xu et al., "Internet of Vehicles in Big Data era," *IEEE/CAA J. Automatica Sinica*, vol. 5, no. 1, pp. 19–35, Jan. 2018.

- [4] M. B. Mollah et al., "Blockchain for the internet of vehicles towards intelligent transportation systems: A survey," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4157–4185, Mar. 2021.
- [5] O. Ghaffarpasand and F. D. Pope, "Telematics data for geospatial and temporal mapping of urban mobility: New insights into travel characteristics and vehicle specific power," *J. Transport Geogr.*, vol. 115, 2024, Art. no. 103815.
- [6] X. Chen et al., "Vehicle as a service (VaaS): Leverage vehicles to build service networks and capabilities for smart cities," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 2048–2081, Third Quarter 2024.
- [7] S. Djahel, R. Doolan, G.-M. Muntean, and J. Murphy, "A communications-oriented perspective on traffic management systems for smart cities: Challenges and innovative approaches," *IEEE Commun. Surveys Tuts.*, vol. 17, no. 1, pp. 125–151, First Quarter 2015.
- [8] J. Wang, C. Jiang, K. Zhang, T. Q. Quek, Y. Ren, and L. Hanzo, "Vehicular sensing networks in a smart city: Principles, technologies and applications," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 122–132, Feb. 2018.
- [9] G. P. R. Filho et al., "Enhancing intelligence in traffic management systems to aid in vehicle traffic congestion problems in smart cities," *Ad Hoc Netw.*, vol. 107, 2020, Art. no. 102265.
- [10] M. L. Han, B. I. Kwak, and H. K. Kim, "Event-triggered interval-based anomaly detection and attack identification methods for an in-vehicle network," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2941–2956, 2021.
- [11] T. Alladi, B. Gera, A. Agrawal, V. Chamola, and F. R. Yu, "Deepadv: A deep neural network framework for anomaly detection in vanets," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12013–12023, Nov. 2021.
- [12] J. Liu, S. Zhang, W. Sun, and Y. Shi, "In-vehicle network attacks and countermeasures: Challenges and future directions," *IEEE Netw.*, vol. 31, no. 5, pp. 50–58, 2017.
- [13] R. A. Bridges, T. R. Glass-Vanderlan, M. D. Iannaccone, M. S. Vincent, and Q. Chen, "A survey of intrusion detection systems leveraging host data," *ACM Comput. Surv.*, vol. 52, no. 6, pp. 1–35, 2019.
- [14] R. Ahmad, I. Alsmadi, W. Alhamdani, and L. Tawalbeh, "Zero-day attack detection: A systematic literature review," *Artif. Intell. Rev.*, vol. 56, no. 10, pp. 10733–10811, 2023.
- [15] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowl.-Based Syst.*, vol. 189, 2020, Art. no. 105124.
- [16] A. Nisioti, A. Mylonas, P. D. Yoo, and V. Katos, "From intrusion detection to attacker attribution: A comprehensive survey of unsupervised methods," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3369–3388, Fourth Quarter 2018.
- [17] L. Yang and A. Shami, "A transfer learning and optimized CNN based intrusion detection system for internet of vehicles," in *Proc. 2022 IEEE Int. Conf. Commun.*, 2022, pp. 2774–2779.
- [18] Y. Yu, X. Zeng, X. Xue, and J. Ma, "LSTM-based intrusion detection system for VANETs: A time series classification approach to false message detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 23906–23918, Dec. 2022.
- [19] B. Cao, C. Li, Y. Song, and X. Fan, "Network intrusion detection technology based on convolutional neural network and BiGRU," *Comput. Intell. Neurosci.*, vol. 2022, no. 1, 2022, Art. no. 1942847.
- [20] C. Fan, J. Cui, H. Jin, H. Zhong, I. Bolodurina, and D. He, "Auto-updating intrusion detection system for vehicular network: A deep learning approach based on cloud-edge-vehicle collaboration," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 15372–15384, Oct. 2024.
- [21] S. Li, Y. Cao, S. Liu, Y. Lai, Y. Zhu, and N. Ahmad, "HDA-IDS: A hybrid dos attacks intrusion detection system for IoT by using semi-supervised CL-GAN," *Expert Syst. Appl.*, vol. 238, 2024, Art. no. 122198.
- [22] S. Das et al., "Network intrusion detection and comparative analysis using ensemble machine learning and feature selection," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4821–4833, Dec. 2022.
- [23] A. Zohourian, S. Dadkhah, H. Molyneaux, E. C. P. Neto, and A. A. Ghorbani, "IoT-prids: Leveraging packet representations for intrusion detection in IoT networks," *Comput. Secur.*, vol. 146, 2024, Art. no. 104034.
- [24] G. Mohi-Ud-Din et al., "Intrusion detection using hybrid enhanced CSA-PSO and multivariate WLS random-forest technique," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4937–4950, Dec. 2023.
- [25] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," Jan. 2023.
- [26] M. A. J. Jamali et al., "A deep neural network-based multi-layer classifier ensembles for intrusion detection in fog-based Internet of Things environments," *Eng. Appl. Artif. Intell.*, vol. 140, 2025, Art. no. 109727.
- [27] W. Wang, Y. Huang, Y. Wang, and L. Wang, "Generalized autoencoder: A neural network framework for dimensionality reduction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops*, 2014, pp. 490–497.
- [28] C. Zhou and R. C. Paffenroth, "Anomaly detection with robust deep autoencoders," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2017, pp. 665–674.
- [29] B. Lampe and W. Meng, "A survey of deep learning-based intrusion detection in automotive applications," *Expert Syst. Appl.*, vol. 221, 2023, Art. no. 119771.
- [30] A. Borghesi, A. Bartolini, M. Lombardi, M. Milano, and L. Benini, "Anomaly detection using autoencoders in high performance computing systems," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 9428–9433.
- [31] S. Hore, Q. H. Nguyen, Y. Xu, A. Shah, N. D. Bastian, and T. Le, "Empirical evaluation of autoencoder models for anomaly detection in packet-based nids," in *Proc. 2023 IEEE Conf. Dependable Secure Comput.*, 2023, pp. 1–8.



Chen Zhang received the B.S. degree in inorganic nonmetallic from the Henan University of Technology, Zhengzhou, China, in 2021, and the master's degree in 2023 from the Computer Science and Engineering, University of Aizu, Aizuwakamatsu, Japan, where he is currently working toward the Ph.D. degree. His research interests mainly include gateway security, information security, and metaverse.



Lu Zheng received the master's degree in 2022 from the Lanzhou University of Technology, Lanzhou, China, where she is currently working toward the Ph.D. degree with the School of Computer and Communication. Her research interests include security analysis and design of communication protocols and formal security evaluation for the Internet of Vehicles.



Huakun Huang (Member, IEEE) received the Ph.D. degree in computer science and engineering from the University of Aizu, Aizuwakamatsu, Japan, in 2019. He is currently an Associate Professor with the School of Computer Science and Cyber Engineering, Guangzhou University, Guangzhou, China. His research interests include privacy preserving, machine learning, federated learning, and continuous intelligent networks.



Chunhua Su (Member, IEEE) received the B.S. degree from Beijing Electronic and Science Institute, Beijing, China, in 2003, and the M.S. and Ph.D. degrees in computer science from the Faculty of Engineering, Kyushu University, Fukuoka, Japan, in 2006 and 2009, respectively. From 2009 to 2011, he was a Postdoctoral Fellow with Singapore Management University, Singapore and Research Scientist with Cryptography & Security Department, Institute for Infocomm Research, Singapore, from 2011 to 2013. From 2013 to 2016, he was an Assistant Professor

with the School of Information Science, Japan Advanced Institute of Science and Technology, Nomi, Japan. From 2016 to 2017, he was an Assistant Professor with the Graduate School of Engineering, Osaka University, Suita, Japan. He is currently a Senior Associate Professor with the Division of Computer Science, University of Aizu, Aizuwakamatsu, Japan. He has authored or coauthored more than 100 papers in international journals and conferences. His research interests include cryptanalysis, cryptographic protocols, privacy-preserving technologies in machine learning, and IoT security & privacy.